Using LLMs on Time Series for Human Activity Recognition (HAR): An Adaptation of AutoTimes

JACQUET Henri
Student, IDIA5
Nantes, France
henri.jacquet@etu.univ-nantes.fr

MASSELOT Théo
Student, IDIA5
Nantes, France
theo.masselot@etu.univ-nantes.fr

WEBER Loïc
Student, IDIA5
Nantes, France
loic.weber@etu.univ-nantes.fr

***Abstract*—Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language processing, yet their application to time series classification (specifically Human Activity Recognition) remains an emerging field of study. This paper investigates the adaptability of the AutoTimes framework, originally designed for forecasting, to perform classification tasks on the WISDM dataset. We propose a modified architecture that leverages frozen pre-trained LLMs coupled with lightweight trainable adapters to process sensor data. Our findings suggest Frozen LLMs architecture provide working classification capabilities for time series.**

***Index Terms*—LLM, Time Series, LLM4TS, AutoTimes, HAR, WISDM, Classification**

I. Introduction

A. Motivation

Large Language Models (LLMs) have gained significant popularity in recent years for processing textual data [1]. These models are pre-trained on vast text corpora, which means they are not inherently optimized for other types of data.

Meanwhile, connected devices have become ubiquitous in many people's daily lives, particularly those used to track physical activities (e.g., jogging, walking, etc.) [2] [3].

B. Context

Our case study builds upon the AutoTimes framework, which leverages a Large Language Model (LLM) for token prediction on time-series data [4].

The time-series data used in this work are sourced from the WISDM dataset [2] [3], which comprises recordings of users performing various activities, including jogging, walking, stair climbing, and others.

C. Objectives

This study pursues the following key objectives:

- To adapt and evaluate the AutoTimes framework for use with the WISDM dataset (including preprocessing and benchmarking) [2], [4].
- To modify the AutoTimes model for human activity classification [4].
- To assess the performance of the classification model [5].
- To propose optimizations and recommendations for improving the model's performance.

II. Prerequisites

A. Time Series

Time series are defined as sequential observations exhibiting diverse characteristics, captured continuously over time [5].

They represent temporal signals generated by both natural phenomena (e.g., meteorological readings) and human activities, including financial records, industrial measurements, and physical activities [2] [3].

B. Auto-Regressive Models

Auto-regression is a fundamental concept shared by both language modeling and some classical statistical forecasting [4].

Unlike recent forecasting models that predict entire sequences in a single step, the auto-regressive approach generates variable-length sequences iteratively.

In the case of LLMs, their inherently auto-regressive nature provides them with superior generalization capabilities and adaptability to diverse tasks [4].

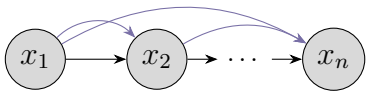

Figure 1. Auto-regressive sequence generation.

C. Large Language Model (LLM) [6]

Large language models typically refer to transformer-based pre-trained language models with billions or more parameters. Autoregressive models predict the next token y based on a given context sequence X, trained by maximizing the probability of the token sequence given the context. Through this, the model achieves intelligent compression and language generation in an autoregressive manner. [6]

III. Related Work

A. *Large Language Model for Time Series (LLM4TS) [6] [7]*

LLM4TS refers to the application of Large Language Models for time series prediction. The core principle leverages LLMs' natural ability to predict next tokens to forecast subsequent points in a time series.

By design, LLMs operate on natural language and are not equipped to process raw time series data. Therefore, time series must be transformed into a format understandable by the model. This transformation enables alignment between temporal data and natural language, **allowing for the incorporation of additional context and information into the original series.**

In the article "What Can Large Language Models Tell Us about Time Series Analysis ?" [6] two main approaches for LLM-based time series prediction are defined: Tuning-based and Non-tuning-based Predictors.

The **Tuning-based Predictor** involves segmenting the time series through patching and tokenizing numerical signals along with related textual data, followed by fine-tuning for time series tasks. However, this method requires modifications that alter the original LLM's parameters, potentially leading to catastrophic forgetting.

The alternative approach, **Non-tuning-based Predictor**, utilizes a pre-trained LLM (already fully functional on text corpora) that remains frozen during time series prediction. To enable the LLM to "understand" time series data (for which it wasn't trained), an **encoder** is typically employed to project the time series into the LLM's latent space (which is accustomed to textual data). A **decoder** is then placed after the model to translate the prediction into a usable format.

This approach offers several advantages: **it eliminates the need to train the model**, resulting in time and cost savings. Another benefit is the model's immediate functionality on unseen datasets. There is little to no training required, and when training is necessary, it focuses on the encoder or decoder rather than the LLM itself.

However, recent studies question the effectiveness of LLMs in these methods, with some even demonstrating improved performance when the LLM is disabled. This is exemplified in the study "Are Language Models Actually Useful for Time Series Forecasting?" [7], where researchers deactivated the LLM component in three recent and popular LLM-based time series forecasting methods. They found that removing the LLM or replacing it with a basic attention layer did not degrade forecasting performance, in most cases, results actually improved.

B. *Deep Neural Networks for Time Series Classification*

In the paper "Time Series Classification from Scratch with Deep Neural Networks: A Strong Baseline" [5], three deep neural network architectures are proposed for time series classification:

1) Multilayer Perceptrons (MLP)
2) Fully Convolutional Networks (FCN)
3) Residual Network (ResNet)

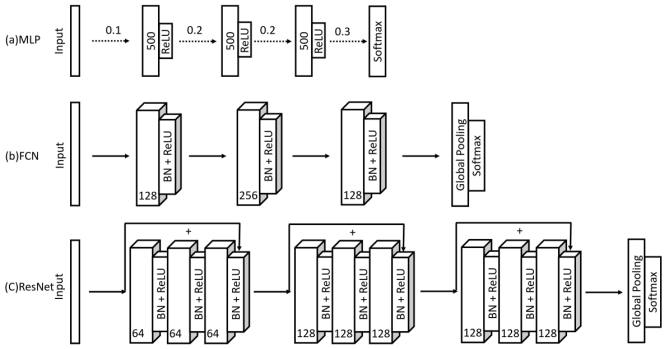

Figure 2. Deep neural network architectures for time series classification [5].

For the Multilayer Perceptron (MLP) architecture, three fully connected layers (FCN) with 500 neurons each are used, each incorporating dropout at the input of every layer to improve generalization. Each of these layers employs a ReLU activation function to prevent gradient saturation in deep networks. The use of ReLU enables deeper network stacking, while dropout helps the model generalize, particularly on small datasets. At the end of the network, a softmax layer is used for classification.

The Fully Convolutional Network (FCN) architecture consists of three convolutional layers, each followed by batch normalization and a ReLU activation function. In this architecture, no pooling operations are applied at each layer to avoid overfitting. Batch normalization is applied after each convolutional layer to accelerate convergence and enhance model generalization. All features are then connected to a Global Average Pooling layer instead of a Fully Connected Layer, significantly reducing the model's parameter count. A softmax layer is used at the end of the network for classification.

For the Residual Network (ResNet) architecture, the FCN blocks from the previous architecture are adapted for ResNet by adding a Global Average Pooling layer followed by a softmax at the end.

To perform time series classification, the authors applied preprocessing, including z-normalization of the data.

C. *AutoTimes [4]*

Unlike conventional non-autoregressive methods that generate predictions in a single forward pass, AutoTimes [4] adopts an **autoregressive generation mechanism**, where predicted tokens are recursively fed back into the model to generate the next predictions. This approach aligns with the inherent architecture of decoder-only LLMs, enabling multi-step forecasting

with arbitrary prediction lengths while preserving the model's general-purpose token transition capabilities.

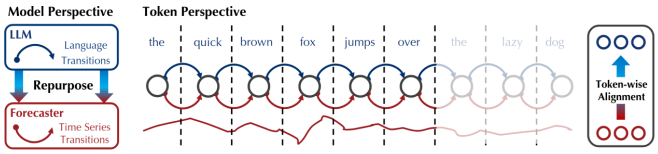

Figure 3. An example to illustrate how AutoTimes adapts language models for time series forecasting. [4].

Autoregressive Time Series Forecasting: AutoTimes repurposes LLMs as autoregressive forecasters by embedding time series segments into the latent space of language tokens. As illustrated in Figure 3, each segment of the time series is treated as a token, and the model predicts the next token based on the preceding ones. This step-by-step prediction method allows the model to forecast for any length of time, just like how language models generate sentences word by word. Unlike other methods, this approach reduces errors that build up over multiple predictions.

Modality Alignment: To connect time series data with the LLM's internal representations, AutoTimes introduces a **SegmentEmbedding** layer that projects time series segments into the embedding space of the LLM. This alignment ensures that the LLM can effectively process time series tokens while maintaining its pre-trained knowledge. AutoTimes also uses text-based timestamps as position markers, helping the model understand chronological order and align different time series variables.

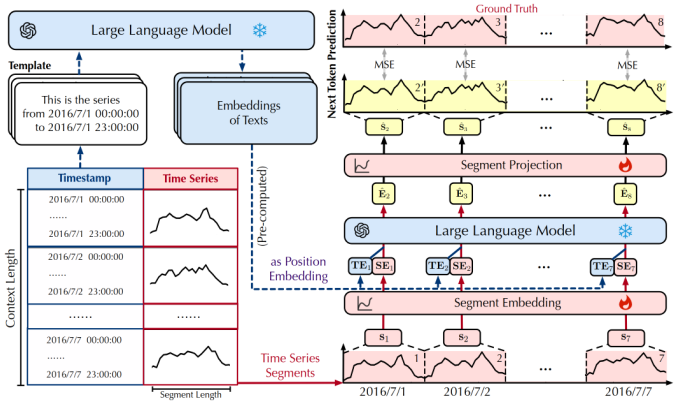

Figure 4. Overview of AutoTimes: (1) time series and corresponding timestamps are segmented; (2) textual timestamps are converted into the position embeddings by the LLM; (3) time series segments are embedded and projected by next token prediction, where intermediate layers of LLM are frozen [4].

In-Context Forecasting: AutoTimes extends the conventional forecasting paradigm by introducing **in-context forecasting**, where time series data itself serves as prompts to guide predictions. Unlike prior methods that rely on natural language prompts, AutoTimes directly uses relevant historical time series segments as context, as shown in Figure 5. This approach enhances the model's ability to generalize to unseen datasets by leveraging domain-specific knowledge embedded in the prompts. The in-context forecasting capability is particularly valuable for zero-shot and few-shot scenarios, where limited or no training data is available for the target domain.

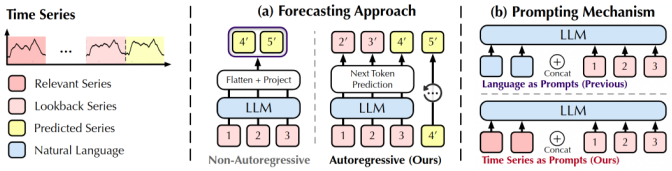

Figure 5. (a) Prevalent LLM4TS methods non-autoregressively generate predictions with the globally flattened representation of lookback series, while large language models inherently predict the next tokens by autoregression. (b) Previous methods adopt language prompts that may lead to the modality disparity, while we find time series can be self-prompted, termed in-context forecasting. [4]

Efficiency and Scalability: AutoTimes achieves remarkable efficiency by freezing the intermediate layers of the LLM and introducing minimal trainable parameters (less than 0.1% of the total model parameters). This lightweight adaptation significantly reduces training and inference time while maintaining state-of-the-art performance.

D. *Dataset*

a) Characteristics: The WISDM dataset [2] [3] was published in 2012 as a Human Activity Recognition (HAR) dataset and was originally presented in a HAR-focused thesis. This dataset contains data from 36 subjects performing **6 different activities**.

b) Limitations: The WISDM dataset [2] [3] explicitly states that the orientation axis for all collected data remains constant.

After further investigation, studies [8] show that the data does not maintain consistent orientations. This creates an inconsistency problem in the WISDM dataset [2].

It is therefore crucial to consider that in WISDM, orientations may vary, which directly affects the training process and consequently the accuracy of results.

In rWISDM [8], which represents the "repaired" version of the WISDM dataset, researchers implemented rules to correct these signals and improve AI training performance.

In our study, we were unable to use rWISDM as the dataset file was not publicly available.

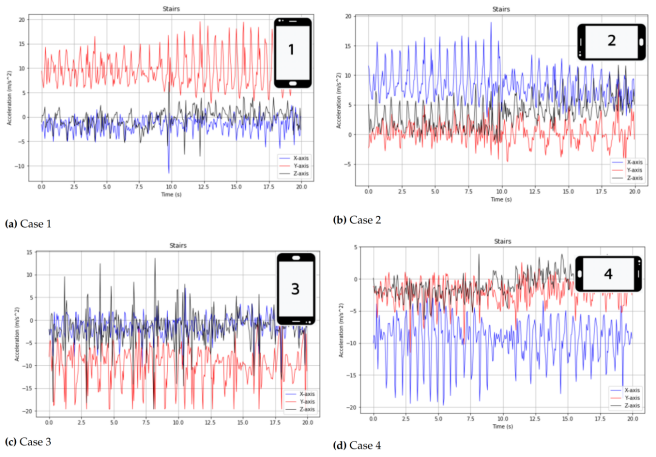

Figure 6. Smartphone-measured accelerations (X, Y, Z axes) according to device orientation. The blue (X), red (Y), and black (Z) curves show acceleration (m/s²) over time. [8].

IV. Prediction

Before adapting AutoTimes [4] for activity classification, we wanted to make sure that the AutoTimes algorithm was functional before modifying it for classification. Furthermore, we wanted to adapt AutoTimes so that we could use the WISDM dataset [2] [3] for time series prediction. In this regard, we have predicted the next 10 points of a time series from the WISDM dataset.

A. *Data Cleaning*

a) Adapting AutoTimes to WISDM: To use the WISDM dataset [2] [3], we first needed to understand its composition. As previously mentioned, the dataset contains data from 36 users performing 6 different activities.

Before using it with AutoTimes, we had to perform data cleaning to make it usable.

We first removed the semicolons (";") at the end of each line and eliminated N/A values.

We then decided to keep only the attributes `["timestamp", "x-axis", "y-axis", "z-axis"]`.

We encountered an issue with the timestamp format, which didn't match what AutoTimes expected. We converted the timestamp type to datetime in nanoseconds.

We chose to normalize the data as this is generally recommended to standardize our dataset.

We split the dataset as follows: 70% for training, 20% for testing, and 10% for validation.

b) Pre-processing Adaptation: By default, for data preprocessing, we provided the LLM with the timestamps from the time series.

For example, for time series k, we would retrieve timestamp k and present the next sentence to the LLM.

B. *First Implementation - Sliding Window Approach*

Our initial approach for segmenting the dataset involved creating sequences of 200 tokens, using 190 tokens as context to predict the next 10 tokens - corresponding to the next 10 points in a time series.

The remaining 10 tokens were used to evaluate prediction performance.

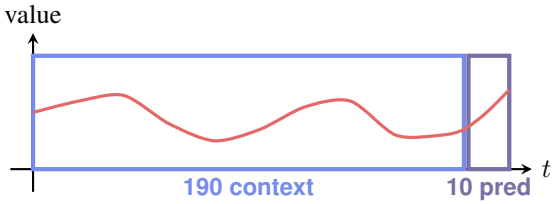

Figure 7. Prediction of the next 10 points from 190 context points in a time series

Our first methodology consisted of creating sliding windows of 200 tokens each across our dataset.

To ensure overlap between time series segments, we implemented a "step" parameter that determines the stride when extracting each time series window.

This stepping mechanism helps maintain continuity across our time series sequences.

For example, with 50,000 data points, this would generate approximately 249 time series segments.

However, this method doesn't account for individual users or activity transitions, which led us to develop another methodology.

C. *Preprocessing Modifications*

In AutoTimes, the preprocess corresponds to the action of preparing the context for the corresponding time series in order to provide it to the LLM.

After further investigation, we found no correlation between timestamp information in the time series and the activity.

Additionally, when converting all timestamps to nanoseconds, we observed that the year and day remained constant, providing no additional useful information to the LLM.

We therefore removed this information, resulting in the LLM receiving the following prompt when processing a time series: `"This is Time Series of (num) points over 10 seconds"`. Previously, this prompt had been formatted as: `"This is Time Series from (start) to (end)"`.

D. *Second Implementation: User- and Activity-Specific Time Series Segmentation*

Before implementing this approach, we performed additional data cleaning by removing all records containing zero values.

Our objective remained to generate time series segments of 200 tokens while following two fundamental constraints:

- Each time series must maintain consistent activity throughout
- Each time series must maintain consistent user throughout

User	Activity	Timestamp	X	Y	Z
33	Jogging	49183874710000	-0.9942854	3.0237172	8.308413
33	Jogging	49183932357000	-1.0760075	3.4459480	8.049625
33	**Jogging**	49184042312000	-1.0760075	5.2165933	6.891896
33	**Walking**	49394992294000	0.84446156	8.0087640	2.7921712

Table I
EXAMPLE OF TIME SERIES OVERLAP FOR A USER PERFORMING TWO DIFFERENT ACTIVITIES.

User	Activity	Timestamp	X	Y	Z
33	Jogging	49183874710000	-0.9942854	3.0237172	8.308413
33	Jogging	49183932357000	-1.0760075	3.4459480	8.049625
34	Jogging	49184042312000	-1.0760075	5.2165933	6.891896

Table II
EXAMPLE OF TIME SERIES SHOWING USER CHANGE DURING A JOGGING ACTIVITY.

This segmentation strategy effectively implements a **"GROUP BY"** operation on both user ID and activity type.

This refined approach yields higher-quality, more precise time series segments. When processing 50,000 data points, we obtained 244 distinct time series. For a dataset of 500,000 points, we generated 2,371 segments - approximately 5% fewer than the theoretical 2,500 segments that would result from a naive sliding window approach that ignores user and activity boundaries.

E. *Prediction Adaptation*

Following our initial sliding window implementation, we modified portions of the AutoTimes codebase to enable prediction of the next 10 points in a time series.

As previously mentioned, our approach involved providing 190 tokens (equivalent to 190 data points) as context to predict the next 10 points.

Adapting AutoTimes for prediction proved to be challenging. The process was complicated by the limited documentation of the existing codebase.

Ultimately, we successfully modified AutoTimes to perform the intended predictions.

V. CLASSIFICATION

For further information about how AutoTimes is implemented, please refer to the **appendices section**. This will allow you to better understand the modifications we made to adapt AutoTimes for classification.

A. *Architecture Adaptation*

To adapt the model for the classification task, we first modified the structure of the target labels. Previously, the target labels (referred to as `batch_y`) contained the ground-truth values of the time series points in each batch, used for training and evaluation. For classification, however, we redefined these labels to represent the class (or activity) of each time series, with a single label per series rather than per time point.

With the target labels now encoding class information, the model's output must correspondingly predict the activity category rather than the next time series point, as was the case before.

This shift represents the **core challenge** of the transformation: **using the same input (i.e., the same batch_x and preprocessing context), the model must now classify time series into categories it has never encountered and for which it has not been explicitly trained**. This is particularly demanding given that **LLMs are inherently unsuited to processing raw time series data**; the encoder and decoder enable this capability. Notably, the LLM's internal response (immediately before the decoder) remains unchanged from its previous state.

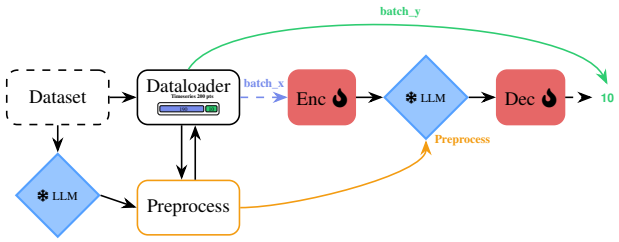

Figure 8. Based AutoTimes architecture (before our adaptation for classification).

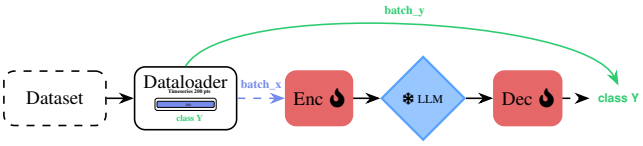

Figure 9. New AutoTimes architecture adapted for classification.

To achieve this, the first step was to modify the decoder's final layer to align with a classification algorithm, specifically to output the predicted activity. We fixed the decoder's final neural layer to 6 units (corresponding to the number of activities) and implemented a softmax activation function to simulate **a probability distribution** over these output neurons. In practice, an explicit softmax layer is unnecessary, as **the**

CrossEntropyLoss function inherently performs this operation, but the theoretical principle remains equivalent.

B. Segmentation Adaptation

We now turn to the unfold transformation, which was briefly mentioned earlier. This transformation is applied to the data immediately before it enters the encoder. The unfold is essentially a segmentation of the time series information, implemented using PyTorch's `tensor.unfold()` function. It extracts multiple slices from the time series, with each slice sized according to the `token_size` parameter. Specifically, unfold generates a number `seq_len // token_len` of slice, meaning the total length of the time series divided by the number of points intended for prediction. Each slice spans `token_len` points, which also corresponds to the input size of the encoder.

The output of this segmentation is then fed into the encoder and subsequently to the LLM. Thus, the LLM receives the encoder's translation of a sequence of multiple slices for each time series, where each slice contains a segment of the original series.

Given this understanding of how unfold (the PyTorch function underlying the fold_out segmentation) processes the data and alters its format, the decoder's output must be adjusted accordingly. Originally, for a single time series, the LLM would return as many predicted points as it received slices, rather than just one. In the initial code, a `tensor.reshape()` operation was applied to the decoder's output to restore the format of batch_y. This reshaping must now be adapted to align with the classification task.

Now that we aim to perform classification, **we obtain as many probability distributions as there are segmentation for each time series.** To address this, we opted **to compute the mean across all these distributions**, yielding the average probability distribution of the slices, and thus the probability distribution of the entire series.

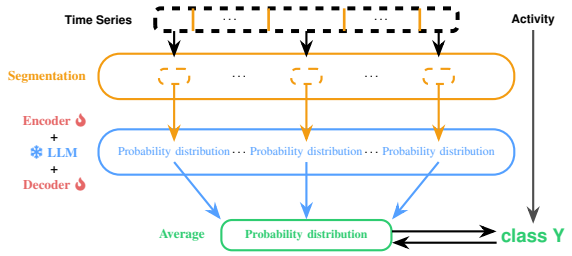

Figure 10. Classification process using segmentation and averaging LLM outputs.

An alternative approach would have been to select only the final probability distribution, derived from the last sliding window of the time series (i.e., the last token_len points). However, we retained the first solution, as it appeared more logical and relevant for our objectives.

We deliberately chose not to remove the segmentation implementation for two key reasons. First, the existing code already functioned effectively, delivering satisfactory performance. Second, removing segmentation would necessitate resizing the encoder to process the entire time series as input, rather than just subsequences. This modification could significantly increase the neural network's size. Additionally, retaining this operation allows us to average the probability distributions, thereby enhancing the reliability of the classification results.

This approach assumes that the patterns we seek (those enabling the classification of time series) are present at sufficiently small scales to be detectable within `token_len` points and that they recur throughout the series with a minimum prevalence of fifty percent among the subsequences (because of the final average operation).

We tested various `token_len` values (5, 10, 50, 100, and 200) and achieved the best performance with `token_len = 10`. This indicates that, for our use case, the relevant patterns are more effectively detected when using smaller subsequences.

The resulting probability distribution can now be compared to the activity labels stored in batch_y, a process consistent with how CrossEntropyLoss operates, as it compares a probability distribution to a class label. The loss can then be computed, and learning can proceed. Subsequently, we implemented classification-specific metrics, namely, accuracy, F1-score, recall, precision and the confusion matrix, to interpret the results of the testing phase.

In summary, we only needed to modify two components of the code: the decoder and the data_loader for batch_y.

C. Preprocess Ablation

To rapidly adjust the data volume, we decided to remove preprocessing for several reasons. First, preprocessing was by far the most time-consuming step in running AutoTimes (e.g., 30 minutes for 50,000 data points, whereas the WISDM dataset contains slightly over 1 million points). Thus, increasing the data volume would have significantly extended processing time. Second, we determined that the contextual information provided by preprocessing added minimal value to the LLM. The generic phrase, "This is a Time Series of (num) points over 10 seconds", was neither personalized for individual time series nor informative enough to justify its computational cost.

However, we do not advocate for the complete elimination of preprocessing. We will revisit this component in the future work section, particularly with proposals for improvement.

D. Data Augmentation

We observed that our results could be explained by the under-representation of certain activities in the dataset. Specifically, activities such as "Sitting," "Standing," and "Downstairs" were never detected by the model, a shortcoming attributable to their low proportional presence in the dataset. In the meantime,

activities like "Walking" and "Jogging", which are heavily represented, were quickly and accurately identified by the model.

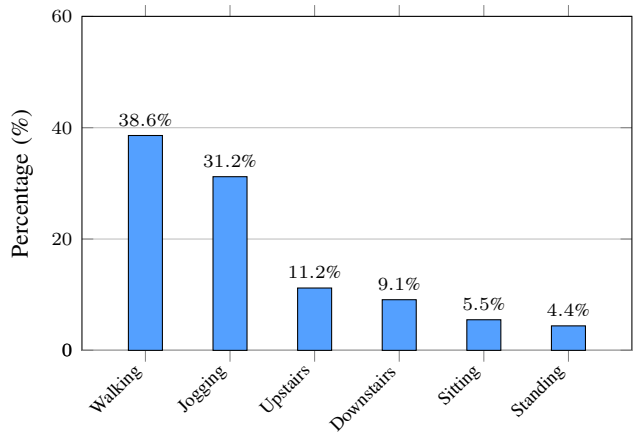

Figure 11. WISDM Activity distribution.

To address this imbalance, we implemented data augmentation. We introduced sliding windows during the creation of time series to increase their overall number, thereby mathematically boosting the representation of initially underrepresented activities. The overlap within each activity–user combination was set to 75%. Importantly, no overlap was introduced across different activity–user combinations.

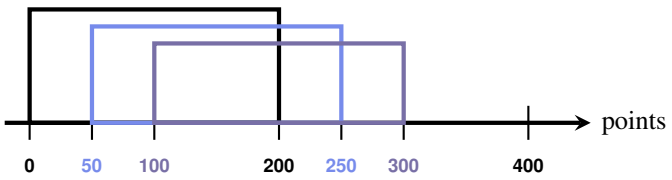

Figure 12. Proportional dataset augmentation

VI. Results and Discussion

To evaluate the effectiveness of the AutoTimes architecture for Human Activity Recognition (HAR), we established a standardized experimental protocol enabling comparison between the LLM-based approach and traditional architectures across varying data regimes.

A. Experimental Setup

- **Dataset**: The WISDM dataset was segmented into windows of 200 points (seq_len).
- **Data Augmentation**: A sliding window strategy with 75% overlap was applied.
- **Optimization**: Training used AdamW (LR: 5×10^{-4}) with a Cosine Annealing scheduler of a 2-epoch warmup and a GeLU activation function.
- **Evaluation**: We tested models at different training data ratios with stratified sampling (100%, 50%, 10%, 5%, and 1%) to assess few-shot learning capabilities.

B. Comparison with SOTA architectures

We began by comparing the performance of our architecture with other classical classification algorithms [3] [9] on the WISDM dataset. Our performance comparison includes a logistic regression, a decision tree (J48), a support vector machine (SVM), and a convolutional neural network (CNN). The first two algorithms were derived from the work of the researchers who originally developed the dataset, while the latter two were sourced from the study "Human Activity Recognition using WISDM Datasets" conducted by researchers. Our results, alongside theirs, are presented in the following table:

Table III
SOTA Performance Comparison

Architecture	Accuracy
SOTA - CNN	**87.85%**
SOTA - J48	85.1%
Exaone-1.2B	84.14%
SOTA - SVM	82.27%
SOTA - Logistic Regression	78.1%

The results indicate that while our architecture performs slightly below some of the other methods, its performance remains largely acceptable. **Thus, a frozen LLM equipped with an encoder and decoder proves to be a working classifier for human activity recognition.**

C. Model Architectures

We compared the following configurations:

- **MLP**: A standard baseline (4 layers, 512 hidden units, ∼539k trainable parameters).
- **Linear Projection**: Encoder → Decoder (no Transformer blocks) (∼34k trainable adapter parameters).
- **Exaone-1.2B**: Encoder → Frozen Exaone-1.2B → Decoder (∼34k trainable adapter parameters).
- **Llama-7B**: Encoder → Frozen Llama-7B → Decoder (∼69k trainable adapter parameters).
- **Gemma3-270M**: Encoder → Frozen Gemma3-270M → Decoder (∼11k trainable adapter parameters).
- **Random Init**: Encoder → Frozen Exaone-1.2B (randomly initialized) → Decoder (∼34k trainable adapter parameters).

D. Performance Benchmarks (Full Data)

Table IV summarizes performance with 100% of the training data (14,454 series). At this scale, **the MLP achieves the highest accuracy (85.67%)**, slightly outperforming the LLM-based models. However, the **LLM-based approaches utilize**

significantly fewer trainable parameters (∼34k vs 539k), acting primarily as frozen feature processors.

Table IV
PERFORMANCE BENCHMARKS (100% DATA)

Architecture	Accuracy	F1-Score	Trainable Params	Time (s)
MLP	**85.67%**	**0.85**	∼539k	**39s**
Exaone-1.2B	84.14%	0.83	∼34k	543s
Llama-7B	81.98%	0.74	∼69k	1322s
Gemma3-270M	80.56%	0.80	**∼11k**	456s

Even if Llama-7B is a larger model than Exaone-1.2B, it performs worse in this task. This can be explained by the fact that Exaone-1.2B is a more recent model than Llama-7B, thus Exaone may have a better architecture.

Table V
COMPARISON OF MODEL ARCHITECTURES

Architecture	Accuracy	F1-Score	Trainable Params	Time (s)
Exaone-1.2B	**84.14%**	**0.83**	∼34k	543
Llama-7B	81.98%	0.74	∼69k	1322
Gemma3-270M	80.56%	0.80	**∼11k**	**456**
Llama-7B (Random Init)	80.62%	0.79	∼69k	1324
Gemma3-270M (Random Init)	79.81%	0.77	**∼11k**	459
Exaone-1.2B (Random Init)	79.54%	0.79	∼34k	559
Linear Projection	40.33%	0.25	∼34k	44s

These results demonstrate that, within our architecture, **the primary classification capability comes from the LLM rather than the supporting layers** (encoder and decoder). This is evident from the performance gap between AutoTimes (Exaone-1.2B) at 84.14% accuracy and the Linear Projection baseline at 40.33%, where the latter lacks the LLM's representational power.

Furthermore, our findings show that **the majority of the recognition performance is actually the consequence of the underlying Transformer architecture of the LLM (decoder-only)** rather than its pretrained knowledge. This is supported by the comparison between Exaone-1.2B (84.14%) and its randomly initialized counterpart (79.54%), **where even a non-pretrained Transformer retains strong classification ability**. This suggests that the structural advantages of the Transformer play a dominant role in feature extraction, while pretraining provides a modest but meaningful performance boost.

E. Few-Shot Learning and Data Efficiency

The primary advantage of the LLM-based approach is revealed in low-data regimes. As shown in Table VI, when training data is reduced to 1% (only 142 samples).

Table VI
FEW-SHOT ACCURACY COMPARISON (%)

Data Ratio	100%	50%	10%	5%	1%
MLP	**85.67**	**83.39**	77.17	74.29	39.42
Exaone-1.2B	84.14	82.71	**78.09**	**76.83**	**65.21**
Exaone (Random Init)	79.54	77.36	70.53	68.81	44.94
Linear Projection	39.41	38.16	42.01	41.20	40.15

At 1% data, the MLP (539k params) drops to 39.42% accuracy, collapsing to majority-class prediction. This highlights the overfitting risk of large trainable parameter counts in few-shot regimes (which could have been avoided by training a MLP with fewer hidden dimensions and hidden layers).

The pretrained Exaone model maintains **65.21%** accuracy at the 1% threshold, a **25.8 percentage point** improvement over the MLP and **20.3 percentage point** improvement over the randomly initialized equivalent. This suggests that the benefits of LLM pre-training become even more pronounced in low-data regimes, where the model's ability to generalize from limited samples is significantly enhanced by its prior knowledge.

This might indicate that **representations learned during language pre-training enable effective few-shot transfer to sensor time series**.

More over, at 1% data, the random Transformer Decoder-only (44.94%) performs only marginally better than the MLP (39.42%), while the pretrained version achieves 65.21%.

F. Limitations and Conclusion

Our results demonstrate that while MLPs are efficient for large labeled datasets, LLM-based architectures provide superior robustness for "cold-start" HAR applications where labeled data is scarce. By freezing pretrained weights and training only lightweight adapters (∼34k parameters), our architecture avoid the catastrophic overfitting scenario observed in standard neural networks with high parameter counts.

VII. KNOWN LIMITATIONS

One of the limitations of this work concerns the WISDM dataset. As previously noted, researchers have identified issues within this dataset. Unfortunately, the corrected version (or the dataset derived from these corrections) is either unavailable or we were unable to locate it, preventing its use in our study.

Additionally, as demonstrated earlier, the distribution of activities in the dataset is suboptimal, which poses challenges for pattern detection, particularly for underrepresented activities. To mitigate this, we employed data augmentation via sliding windows, but we believe further improvements in this area are still possible. We also attempted a more refined data augmentation approach, specifically, SMOTE (Synthetic Minority Over-sampling Technique) to balance the activity distribution. However, this did not yield performance improvements. We hypothesize that this may come from the complexity of the

activities themselves, with patterns that are too subtle for an LLM (especially one not fine-tuned for this task) to reliably identify.

VIII. Future Work

Numerous avenues for future research exist to extend our study. Several key directions include:

First, the generalization of our findings to other classification datasets, and, more broadly, to domains within time series classification beyond human activity recognition (HAR).

Another promising direction involves exploring additional LLMs beyond the three models we utilized (Llama-7B, Exaone-1.2B, and Gemma3-270M). Specifically, investigating larger models could reveal their ability to identify more complex patterns, particularly in scenarios with limited training data.

A further avenue is the trial of "SequenceClassification" model. In our work, we employed "CausalLM" model from Hugging Face's Transformers library, as it was the default used in AutoTimes. While this choice was logical for autoregressive prediction tasks, it may be less suited for classification. In contrast, "SequenceClassification", also available in the Transformers library, are specifically designed for sequence classification, precisely aligning with our use case.

It will also be relevant to further investigate the parameters seq_len and token_len. While these parameters inherently depend on the time series under study and the classification target, establishing a framework to identify optimal parameter combinations would be highly valuable.

Relatedly, it would be insightful to explore removing the segmentation operation entirely and feeding the model the time series in its entirety (i.e., unsliced), particularly if the patterns of interest are suspected to span many data points.

In parallel, transforming the unfold operation into an information concentration mechanism, like max-pooling or average pooling in CNNs, could offer significant benefits. For instance, this could involve condensing the information from 20 points in the original series into just 10, thereby encapsulating more information within fewer points.

Additionally, refining the supporting layers (the encoder and decoder) warrants further exploration. While our best results were achieved with a simple linear network architecture, more complex networks, as used in AutoTimes, might yield advantages. It could also be interesting to vary the encoder and decoder independently, as they are currently constrained by a symmetrical linkage.

Reintegrating preprocessing is a critical avenue for further exploration, as preprocessing plays a pivotal role in AutoTimes and significantly enhances performance. To reactivate it, we propose two approaches. The first involves processing all dataset entries individually and storing the results before use, as previously implemented. However, this method is time-consuming and resource intensive. The second approach entails generating a generic contextual phrase for all data points, which is logical for our WISDM dataset, as discussed earlier. This generic phrase would be easy to generate, store, and retrieve. It would also allow for straightforward experimentation with different phrasing structures to identify the most effective variant. Potential examples include: "This is a time series of an unknown activity, comprising (num) points over 10 seconds" or "This is a time series of an activity among Walking, Jogging, ..., comprising (num) points over 10 seconds. Can you guess the activity?"

Finally, the most promising direction may involve replacing the Transformer Decoder-only architecture with a Transformer Encoder-only model (whether trained on natural language or not). We hypothesize that the architecture of an Encoder-only Transformer (which remains frozen here) will be even more effective than a Decoder-only model in identifying complex patterns within time series data.

IX. Conclusion

We successfully adapted AutoTimes for time series classification in the domain of human activity recognition (HAR). Our findings demonstrate that an architecture leveraging a frozen LLM, with only a limited number of trainable parameters, do work as an effective classifier, though it remains less performant than specialized time series classification algorithms.

Our analysis revealed that, within this architecture, performance primarily come from the LLM itself rather than the supporting layers (encoder and decoder). Although LLM performs less well than other algorithms when trained on large datasets, we have observed that its performance degrades less rapidly than some other classifiers architectures as the volume of training data decreases. This resilience may makes it a usable classification option in resource- constrained environments.

Furthermore, we demonstrated that the performance of LLMs primarily results from their inherent architecture, that of a Transformer Decoder-only model. Indeed, even without natural language training, these models exhibit reasonable performance. While pretraining on natural language enhances their pattern detection capabilities to some extent, this improvement becomes particularly pronounced when the volume of training data is limited.

Finally, we hypothesize that, with effective projection methods, a single LLM could serve as a robust pattern detector across multiple domains. The concept of an LLM as a feature extractor warrants further investigation.

References

[1] C. Liu, H. Miao, C. Long, Y. Zhao, Z. Li, and P. Kalnis, "Llms meet cross-modal time series analytics: Overview and directions," 2025.

[2] W. Lab, "Wisdm smartphone and smartwatch activity and biometrics dataset," http://www.cis.fordham.edu/wisdm/dataset.php, 2012.

[3] J. R. Kwapisz, G. M. Weiss, and S. A. Moore, "Activity recognition using cell phone accelerometers," 2010.

[4] Y. Liu, G. Qin, X. Huang, J. Wang, and M. Long, "Autotimes: Autoregressive time series forecasters via large language models," 2024.

[5] W. Y. Zhiguang Wang and T. Oates, "Time series classification from scratch with deepneural networks: A strong baseline," 2025.

[6] M. Jin, Y. Zhang, W. Chen, K. Zhang, Y. Liang, B. Yang, J. Wang, S. Pan, and Q. Wen, "Position: What can large language models tell us about time series analysis," 2024.

[7] Tan, Mingtian, Merrill, M. A, Gupta, Vinayak, Althoff, Tim, Hartvigsen, and Thomas, "Are language models actually useful for time series forecasting?" 2024.

[8] M. Heydarian and T. E. Doyle, "rwisdm: Repaired wisdm, a public dataset for human activity recognition," 2023.

[9] P. Seelwal1, A. P. N2, C. Srinivas3, and R. V. S4, "Human activity recognition using wisdm datasets," 2023.

Appendix

Appendix A
Understanding implementation

To modify AutoTimes, we first needed to thoroughly understand its Python implementation in order to identify the necessary changes and avoid a complete rewrite. By that point, we had already adapted the dataset reading and loading processes to align with a classification task.

The code operates as follows:

The data is retrieved from the dataset using a data loader, and time series are generated, all of uniform length (seq_len), which is set as a parameter. The points of each time series are stored in a tensor containing the n time series of the batch, with each point represented by its three coordinates: x, y, and z. Thus, there is a tensor, batch_x, which comprises 16 time series of 190 points each, with three dimensions per point (where 16 is the batch_size and 190 is seq_len – token_len).

Once the data is retrieved and stored in batch_x, prediction is initiated. To do this, transformations are applied to the data, specifically, normalization and segmentation of the tensor, before feeding it into the encoder. The encoder's output, which matches the size of the model's hidden state, is then passed to the LLM, which performs the prediction task. These predictions are subsequently fed into the decoder, and the prior normalization operation is reversed. The model's prediction is thus obtained in natural language. Once these predictions are obtained (x predictions per time series, with the number of time series in the batch), they are compared to the ground truth, which is stored in batch_y and was retrieved simultaneously with batch_x.

We now turn to the topic of preprocessing. Preprocessing involves providing the LLM with additional contextual information to improve prediction accuracy. To achieve this, for each data point in a time series (represented in three dimensions), a contextual sentence is generated. This sentence encapsulates the specific context of the time series to which the data point belongs. The sentence is then provided to the LLM and stored in a file at the same index as the corresponding data point. Each sentence is thus tailored to its respective time series.

When AutoTimes is executed, as the data is loaded into batch_x and batch_y, the contextual sentence is retrieved from the storage file. If a specific AutoTimes parameter is enabled, this sentence can be appended to the batch_x data after it has passed through the encoder. The LLM then utilizes both the contextual sentence and the data for each time series, enhancing its predictive capability.

Batches proceed sequentially until the end of the epoch, at which point the cost function is computed over the validation set to assess whether overfitting has occurred. Once training is complete, the algorithm transitions to the prediction, or testing, phase, as referred to in the code.

It is crucial to understand that, in AutoTimes, the training process does not involve learning a prediction (or classification) algorithm per se. Instead, it focuses on training a translation model. The encoder and decoder are neural networks (of varying depths) designed to enable the LLM to better represent the time series for prediction (or classification) tasks, as LLMs are inherently trained to process natural language text rather than raw time series data. Indeed, the LLM itself, kept frozen, is responsible for performing the prediction or classification. The entire surrounding architecture serves to optimize the conditions under which the LLM operates, ensuring it is best positioned to carry out this task effectively.